Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Высшая школа кибертехнологий, математики и статистики
Кафедра прикладной информатики и информационной безопасности

ТВОРЧЕСКОЕ ЗАДАНИЕ

по дисциплине «Базы данных»

на тему: «Разработка базы данных фитнес-клуба»

Выполнил: студент группы 15.27Д-ИСТ03/24б
Докучаев Михаил Сергеевич

Руководитель:
должность, учёная степень, Фамилия И.О.

Москва — 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
ГЛАВА 1. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ.....4
1.1 Описание предметной области.....4
1.2 Концептуальная модель предметной области.....5
1.3 Даталогическая реляционная модель базы данных.....7
ГЛАВА 2. РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ.....12
2.1 Сценарий создания объектов базы данных.....12
2.2 SQL-скрипт заполнения таблиц данными.....13
2.3 SQL-запросы. Команда SELECT.....14
2.4 Триггеры и хранимые процедуры.....20
ЗАКЛЮЧЕНИЕ.....23
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ.....24
Приложение А. Сценарий создания объектов базы данных.....25
Приложение Б. Скрипт заполнения таблиц тестовыми данными.....29
Приложение В. Триггеры и хранимые процедуры.....32

ВВЕДЕНИЕ

Фитнес-клуб каждый день работает с большим количеством однотипных записей: кто купил абонемент, когда он заканчивается, кто пришёл на тренировку, сколько свободных мест осталось на занятии. Пока клиентов несколько десятков, такой учёт можно вести в тетради или в электронной таблице. Когда их становится несколько сотен, появляются ошибки: в клуб проходят по просроченным картам, на групповое занятие записывают больше людей, чем вмещает зал, а данные о выручке приходится собирать вручную.

Эти задачи решает база данных. В ней сведения хранятся один раз и в одном месте, связи между ними контролирует сама СУБД, а нужные отчёты получаются запросами. Поэтому разработка базы данных для фитнес-клуба остаётся практически значимой задачей. Предметная область удобна и для учебного проекта: в ней есть справочники, связи разных типов и бизнес-правила, которые реализуются триггерами.

Цель работы — спроектировать и реализовать реляционную базу данных фитнес-клуба, в которой ведётся учёт абонементов, посещений, тренеров, расписания занятий и оборудования.

Для достижения цели решаются следующие задачи:

– описать предметную область и требования к хранимым данным;

– построить концептуальную модель: выделить сущности, их атрибуты, связи и домены;

– преобразовать концептуальную модель в даталогическую реляционную модель и описать таблицы;

– написать сценарий создания объектов базы данных в СУБД MySQL;

– заполнить таблицы тестовыми данными;

– составить запросы на выборку данных семи типов, предусмотренных заданием;

– реализовать бизнес-правила с помощью триггеров и хранимых процедур и проверить их работу.

Объект исследования — деятельность фитнес-клуба по обслуживанию клиентов. Предмет исследования — структура данных, необходимых для учёта этой деятельности, и способы их обработки средствами языка SQL.

База данных реализована в СУБД MySQL версии 8.0. Эта система распространяется свободно, входит в число самых используемых реляционных СУБД [14] и подробно документирована [15]. Модель строилась и запросы выполнялись в среде MySQL Workbench [16]. При проектировании применялись подходы из учебной литературы [4, 5, 7, 8].

Работа состоит из введения, двух глав, заключения, списка литературы и трёх приложений.

ГЛАВА 1. ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ

1.1 Описание предметной области

Рассматривается фитнес-клуб с одним филиалом. В клубе есть тренажёрный зал, залы для групповых занятий, кардиозона, зал единоборств и бассейн. Клиенты занимаются самостоятельно, посещают групповые тренировки по расписанию или берут персональные тренировки у тренера.

Клиент получает доступ в клуб, купив абонемент. Каждый абонемент оформляется по одному из тарифов. Тариф задаёт срок действия в днях, цену, допустимое число посещений (для безлимитных тарифов ограничения нет) и число дней, на которое абонемент можно заморозить. При продаже клиенту выдаётся клубная карта с уникальным номером. У одного клиента со временем накапливается несколько абонементов: закончился один, куплен следующий.

Каждый проход в клуб по карте фиксируется как посещение: записываются время входа и время выхода. Пока клиент находится в клубе, время выхода неизвестно.

Групповые и персональные тренировки проводятся по расписанию. Строка расписания определяет, какая тренировка, в каком зале, каким тренером и во сколько проводится, а также сколько на ней мест. Клиент записывается на занятие заранее; запись можно отменить. После занятия администратор отмечает, пришёл ли клиент.

За залами закреплено оборудование. У каждой единицы есть инвентарный номер, дата покупки и состояние: исправно, в ремонте или списано.

С базой данных работают четыре группы пользователей. Администратор на стойке регистрирует клиентов, продаёт абонементы, отмечает посещения и записывает на занятия. Менеджер составляет расписание и ведёт справочники тарифов и тренировок. Тренер просматривает своё расписание и списки записавшихся. Управляющий получает сводные отчёты о продажах, посещаемости и загрузке тренеров.

В результате обследования сформулированы требования к данным и бизнес-правила:

– о клиенте хранятся фамилия, имя, отчество, дата рождения, пол, телефон, адрес электронной почты и дата регистрации; телефон у каждого клиента свой;

– номер клубной карты уникален;

– дата окончания абонемента не может быть раньше даты начала;

– абонемент находится в одном из трёх состояний: активен, заморожен, истёк;

– проход в клуб разрешён только по активному абонементу в пределах срока его действия и лимита посещений;

– в одном зале в одно время проводится не больше одного занятия, и тренер не может вести два занятия одновременно;

– число записей на занятие не превышает числа мест;

– на одно занятие клиент записывается один раз;

– записаться на занятие может только клиент, у которого на дату занятия есть активный абонемент;

– заморозить абонемент можно не больше чем на число дней, предусмотренное тарифом.

Данные из базы используются в нескольких документах клуба. Договор с клиентом и клубная карта содержат сведения о клиенте, тарифе, сроке действия и оплаченной сумме. Расписание на неделю печатается для стойки администратора и сайта. Журнал посещений нужен для контроля загрузки залов. Инвентарная ведомость содержит перечень оборудования по залам с его состоянием.

Сведения о клиентах относятся к персональным данным, поэтому их хранение и обработка должны соответствовать требованиям федерального закона «О персональных данных» [1]. В базе хранится только тот минимум сведений, который нужен для обслуживания клиента.

1.2 Концептуальная модель предметной области

Концептуальная модель описывает предметную область в терминах сущностей и связей и не зависит от конкретной СУБД [8, 9]. Термины «база данных», «модель данных» и «домен» используются в значениях, принятых в стандарте [2] и во вводном курсе С. Д. Кузнецова [10]. Анализ требований позволил выделить десять сущностей. Их перечень приведён в таблице 1.

Таблица 1 — Сущности предметной области

Сущность	Описание	Идентификатор	Основные атрибуты
Тариф (membership_type)	Вид абонемента с условиями	Код тарифа	Название, срок, лимит посещений, цена, дни заморозки
Клиент (client)	Человек, зарегистрированный в клубе	Код клиента	ФИО, дата рождения, пол, телефон, почта, дата регистрации
Абонемент (membership)	Проданный клиенту абонемент	Код абонемента	Номер карты, даты начала и окончания, оплата, статус
Посещение (visit)	Проход в клуб по карте	Код посещения	Время входа, время выхода
Тренер (trainer)	Сотрудник, проводящий занятия	Код тренера	ФИО, специализация, телефон, дата приёма, ставка
Зал (hall)	Помещение клуба	Код зала	Название, вместимость, площадь

Сущность	Описание	Идентификатор	Основные атрибуты
Оборудование (equipment)	Единица инвентаря	Код оборудования	Название, инвентарный номер, дата покупки, состояние
Тренировка (workout)	Вид занятия	Код тренировки	Название, вид, длительность, доплата
Занятие (schedule)	Строка расписания	Код занятия	Время начала, число мест
Запись (booking)	Запись клиента на занятие	Код записи	Время записи, статус

Из таблицы видно, что у каждой сущности в качестве идентификатора выбран суррогатный ключ — числовой код. Естественные ключи (телефон, номер карты, инвентарный номер) могут измениться, поэтому они оформлены как ограничения уникальности, а не как первичные ключи.

Связи между сущностями приведены в таблице 2. Все они имеют тип «один ко многим».

Таблица 2 — Связи между сущностями

Родительская сущность	Дочерняя сущность	Смысл связи
Тариф	Абонемент	По одному тарифу продаётся много абонементов
Клиент	Абонемент	Клиент покупает несколько абонементов
Абонемент	Посещение	По абонементу совершается много посещений
Тренировка	Занятие	Одна тренировка ставится в расписание много раз
Тренер	Занятие	Тренер проводит много занятий
Зал	Занятие	В зале проходит много занятий
Зал	Оборудование	В зале размещено много единиц оборудования
Клиент	Запись	Клиент делает много записей
Занятие	Запись	На занятие записывается много клиентов

По таблице видно, что связь «многие ко многим» между клиентом и занятием разложена на две связи «один ко многим» через сущность «Запись». Так же устроена сущность «Занятие»: она связывает тренировку, тренера и зал. Все связи неидентифицирующие, потому что у дочерних сущностей есть собственные идентификаторы.

Для атрибутов с одинаковым смыслом определены домены — общие множества допустимых значений. В MySQL нет отдельной команды создания домена, поэтому домен реализуется одинаковым типом данных и ограничениями в каждом столбце, который к нему относится. Домены описаны в таблице 3.

Таблица 3 — Домены атрибутов

Домен	Тип данных MySQL	Ограничения	Где используется
Идентификатор	INT UNSIGNED	Автоинкремент, не пусто	Первичные и внешние ключи
Имя человека	VARCHAR(50)	Не пусто (кроме отчества)	ФИО клиентов и тренеров
Телефон	CHAR(12)	Формат +7XXXXXXXXXX, уникален	Клиент, тренер
Денежная сумма	DECIMAL(10,2), DECIMAL(8,2)	Не меньше нуля	Цены, оплата, ставка
Дата	DATE	Не пусто	Рождение, регистрация, сроки
Момент времени	DATETIME	—	Вход, выход, начало занятия
Пол	ENUM('М','Ж')	Одно из двух значений	Клиент
Статус абонемента	ENUM	активен, заморожен, истёк	Абонемент
Статус записи	ENUM	записан, посетил, отменил	Запись
Состояние	ENUM	исправно, в ремонте, списано	Оборудование
Количество	SMALLINT UNSIGNED	Больше нуля	Места, вместимость, дни

Из таблицы следует, что для денежных сумм выбран точный тип DECIMAL, а не числа с плавающей точкой: при сложении цен не возникает ошибок округления. Перечислимые значения заданы типом ENUM, поэтому записать в столбец статус, которого нет в списке, невозможно.

1.3 Даталогическая реляционная модель базы данных

Даталогическая модель получена из концептуальной по обычным правилам: каждая сущность стала таблицей, атрибуты — столбцами, идентификаторы — первичными ключами, а каждая связь «один ко многим» — внешним ключом в дочерней таблице [7, 13]. Модель построена в среде MySQL Workbench в нотации IDEF1X и показана на рисунке 1. Пунктирные линии обозначают неидентифицирующие связи, точка стоит на стороне «многие».

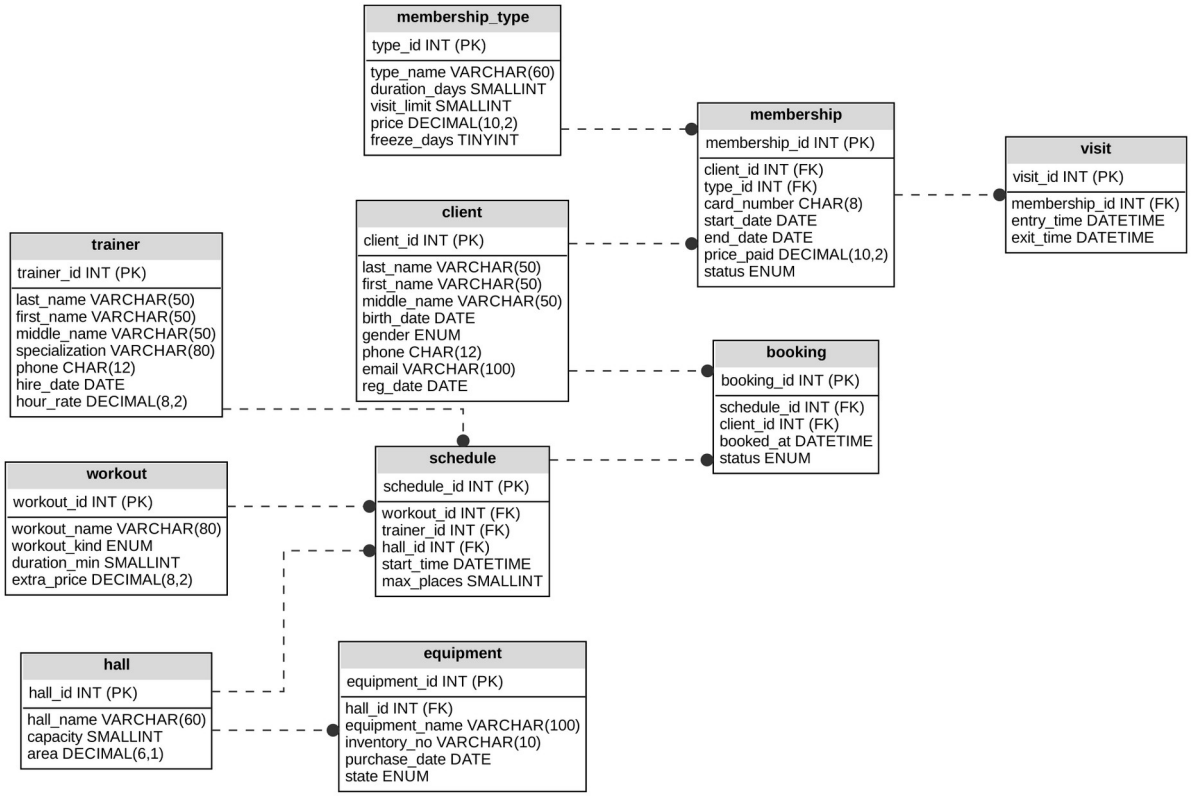


Рисунок 1 — Даталогическая модель базы данных фитнес-клуба (выполнен студентом Докучаевым М.С. с использованием программного продукта MySQL Workbench)

Замените эту схему своим скриншотом из MySQL Workbench: Database → Reverse Engineer → выбрать базу fitness_club. Схема выше построена автоматически по тому же скрипту и показывает те же таблицы и связи.

Все таблицы находятся в третьей нормальной форме [6]. Значения атрибутов атомарны, каждый неключевой столбец зависит от всего первичного ключа и не зависит от других неключевых столбцов. Например, цена тарифа хранится в таблице тарифов, а в абонементе записана фактически уплаченная сумма: она может отличаться из-за скидки и не должна меняться при изменении прайса.

Структура таблиц приведена в таблицах 4–13. В столбце «Ключ» приняты обозначения: PK — первичный ключ, FK — внешний ключ, UQ — ограничение уникальности.

Таблица 4 — Структура таблицы membership_type (тарифы)

Столбец	Тип данных	Ключ	Описание
type_id	INT UNSIGNED	PK	Код тарифа
type_name	VARCHAR(60)	UQ	Название тарифа
duration_days	SMALLINT UNSIGNED	—	Срок действия, дней
visit_limit	SMALLINT UNSIGNED	—	Лимит посещений; NULL — без ограничения

Столбец	Тип данных	Ключ	Описание
price	DECIMAL(10,2)	—	Цена по прайсу, руб.
freeze_days	TINYINT UNSIGNED	—	Допустимая заморозка, дней

Единственный необязательный столбец таблицы — visit_limit: пустое значение означает безлимитный тариф.

Таблица 5 — Структура таблицы client (клиенты)

Столбец	Тип данных	Ключ	Описание
client_id	INT UNSIGNED	PK	Код клиента
last_name	VARCHAR(50)	—	Фамилия
first_name	VARCHAR(50)	—	Имя
middle_name	VARCHAR(50)	—	Отчество (необязательно)
birth_date	DATE	—	Дата рождения
gender	ENUM('М','Ж')	—	Пол
phone	CHAR(12)	UQ	Телефон
email	VARCHAR(100)	UQ	Электронная почта (необязательно)
reg_date	DATE	—	Дата регистрации в клубе

Телефон и адрес почты уникальны, поэтому одного человека нельзя завести в базе дважды.

Таблица 6 — Структура таблицы membership (абонементы)

Столбец	Тип данных	Ключ	Описание
membership_id	INT UNSIGNED	PK	Код абонемента
client_id	INT UNSIGNED	FK	Клиент (client)
type_id	INT UNSIGNED	FK	Тариф (membership_type)
card_number	CHAR(8)	UQ	Номер клубной карты
start_date	DATE	—	Дата начала действия
end_date	DATE	—	Дата окончания действия
price_paid	DECIMAL(10,2)	—	Оплаченная сумма, руб.
status	ENUM	—	Статус: активен, заморожен, истёк

Таблица связывает клиента с тарифом двумя внешними ключами; проверочное ограничение не допускает дату окончания раньше даты начала.

Таблица 7 — Структура таблицы visit (посещения)

Столбец	Тип данных	Ключ	Описание
visit_id	INT UNSIGNED	PK	Код посещения
membership_id	INT UNSIGNED	FK	Абонемент (membership)
entry_time	DATETIME	—	Время входа
exit_time	DATETIME	—	Время выхода; NULL, пока клиент в клубе

Посещение ссылается на абонемент, а не на клиента: так видно, по какой именно карте был проход.

Таблица 8 — Структура таблицы trainer (тренеры)

Столбец	Тип данных	Ключ	Описание
trainer_id	INT UNSIGNED	PK	Код тренера
last_name	VARCHAR(50)	—	Фамилия
first_name	VARCHAR(50)	—	Имя
middle_name	VARCHAR(50)	—	Отчество (необязательно)
specialization	VARCHAR(80)	—	Специализация
phone	CHAR(12)	UQ	Телефон
hire_date	DATE	—	Дата приёма на работу
hour_rate	DECIMAL(8,2)	—	Ставка за час, руб.

Структура таблицы тренеров близка к таблице клиентов, но хранится отдельно: у тренера другие атрибуты и другие связи.

Таблица 9 — Структура таблицы hall (залы)

Столбец	Тип данных	Ключ	Описание
hall_id	INT UNSIGNED	PK	Код зала
hall_name	VARCHAR(60)	UQ	Название зала
capacity	SMALLINT UNSIGNED	—	Вместимость, человек
area	DECIMAL(6,1)	—	Площадь, кв. м

Таблица залов является справочником: на неё ссылаются расписание и оборудование.

Таблица 10 — Структура таблицы equipment (оборудование)

Столбец	Тип данных	Ключ	Описание
equipment_id	INT UNSIGNED	PK	Код оборудования
hall_id	INT UNSIGNED	FK	Зал (hall)
equipment_name	VARCHAR(100)	—	Название
inventory_no	VARCHAR(10)	UQ	Инвентарный номер
purchase_date	DATE	—	Дата покупки
state	ENUM	—	Состояние: исправно, в ремонте, списано

Каждая единица оборудования закреплена за одним залом и однозначно определяется инвентарным номером.

Таблица 11 — Структура таблицы workout (виды тренировок)

Столбец	Тип данных	Ключ	Описание
workout_id	INT UNSIGNED	PK	Код тренировки
workout_name	VARCHAR(80)	UQ	Название

Столбец	Тип данных	Ключ	Описание
workout_kind	ENUM	—	Вид: групповая, персональная
duration_min	SMALLINT UNSIGNED	—	Длительность, мин (от 15 до 240)
extra_price	DECIMAL(8,2)	—	Доплата сверх абонемента, руб.

Большинство тренировок входит в абонемент, поэтому доплата по умолчанию равна нулю.

Таблица 12 — Структура таблицы schedule (расписание занятий)

Столбец	Тип данных	Ключ	Описание
schedule_id	INT UNSIGNED	PK	Код занятия
workout_id	INT UNSIGNED	FK	Тренировка (workout)
trainer_id	INT UNSIGNED	FK, UQ	Тренер (trainer)
hall_id	INT UNSIGNED	FK, UQ	Зал (hall)
start_time	DATETIME	UQ	Дата и время начала
max_places	SMALLINT UNSIGNED	—	Число мест

Два составных ограничения уникальности — (зал, время) и (тренер, время) — не дают поставить два занятия в один зал или одному тренеру на одно время.

Таблица 13 — Структура таблицы booking (записи на занятия)

Столбец	Тип данных	Ключ	Описание
booking_id	INT UNSIGNED	PK	Код записи
schedule_id	INT UNSIGNED	FK, UQ	Занятие (schedule)
client_id	INT UNSIGNED	FK, UQ	Клиент (client)
booked_at	DATETIME	—	Когда сделана запись
status	ENUM	—	Статус: записан, посетил, отменил

Пара (занятие, клиент) уникальна, поэтому записаться на одно занятие дважды нельзя.

Для всех внешних ключей задано каскадное обновление. Удаление родительской строки, на которую есть ссылки, запрещено: нельзя удалить клиента, у которого есть абонементы, или зал, в котором стоят занятия. Исключение сделано для записей на занятие: при удалении занятия из расписания его записи удаляются вместе с ним.

ГЛАВА 2. РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ

2.1 Сценарий создания объектов базы данных

База данных создана в СУБД MySQL версии 8.0 с кодировкой utf8mb4, которая корректно хранит русский текст. Все таблицы используют механизм хранения InnoDB: он поддерживает внешние ключи и транзакции [11, 15]. Сценарий создаёт базу данных, десять таблиц, четыре индекса и представление. Полный текст сценария приведён в приложении А, ниже разобраны его основные части.

Сначала создаётся сама база данных. Команда DROP DATABASE IF EXISTS позволяет запускать сценарий повторно.

```
DROP DATABASE IF EXISTS fitness_club;
CREATE DATABASE fitness_club
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE fitness_club;
```

Таблицы создаются в порядке от родительских к дочерним, иначе внешний ключ не на что будет сослаться. Первой идёт таблица тарифов, затем клиенты и абонементы. Ниже приведена команда создания таблицы абонементов: в ней есть первичный ключ, ограничение уникальности, два внешних ключа и два проверочных ограничения.

```
CREATE TABLE membership (
  membership_id INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  client_id     INT UNSIGNED  NOT NULL,
  type_id       INT UNSIGNED  NOT NULL,
  card_number   CHAR(8)       NOT NULL,
  start_date    DATE          NOT NULL,
  end_date      DATE          NOT NULL,
  price_paid    DECIMAL(10,2) NOT NULL,
  status        ENUM('активен','заморожен','истёк')
                              NOT NULL DEFAULT 'активен',
  CONSTRAINT pk_membership PRIMARY KEY (membership_id),
  CONSTRAINT uq_membership_card UNIQUE (card_number),
  CONSTRAINT fk_membership_client FOREIGN KEY (client_id)
    REFERENCES client (client_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_membership_type FOREIGN KEY (type_id)
    REFERENCES membership_type (type_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_membership_dates CHECK (end_date >= start_date),
  CONSTRAINT chk_membership_price CHECK (price_paid >= 0)
) ENGINE = InnoDB;
```

Остальные таблицы создаются аналогично. В таблице расписания два составных ограничения уникальности реализуют правило о том, что зал и тренер не могут быть заняты дважды в одно время. Первичные ключи и ограничения уникальности MySQL индексирует автоматически. Дополнительно созданы четыре индекса по столбцам, которые часто

встречаются в условиях поиска: фамилия клиента, дата окончания абонемента, время начала занятия и время входа.

```
CREATE INDEX idx_client_name     ON client (last_name, first_name);
CREATE INDEX idx_membership_end  ON membership (end_date, status);
CREATE INDEX idx_schedule_start  ON schedule (start_time);
CREATE INDEX idx_visit_entry     ON visit (entry_time);
```

Последним создаётся представление v_schedule_full. Оно собирает расписание в читаемом виде: вместо кодов тренировки, тренера и зала показывает их названия. Представление работает как сохранённый запрос: данные в нём не хранятся, а каждый раз берутся из таблиц.

```
CREATE VIEW v_schedule_full AS
SELECT s.schedule_id,
       s.start_time,
       w.workout_name,
       w.workout_kind,
       CONCAT(t.last_name, ' ', t.first_name) AS trainer_name,
       h.hall_name,
       s.max_places
FROM schedule s
  INNER JOIN workout w ON w.workout_id = s.workout_id
  INNER JOIN trainer t ON t.trainer_id = s.trainer_id
  INNER JOIN hall    h ON h.hall_id    = s.hall_id;
```

2.2 SQL-скрипт заполнения таблиц данными

Таблицы заполнены тестовыми данными командами INSERT. Данные вымышленные, но согласованы между собой: посещения совершены по действующим на тот момент абонементам, записи сделаны на занятия из расписания, а время посещений совпадает с временем занятий. Порядок заполнения тот же, что и при создании: сначала справочники, затем зависимые таблицы. Полный текст скрипта приведён в приложении Б.

Пример заполнения справочника тарифов:

```
INSERT INTO membership_type
  (type_id, type_name, duration_days, visit_limit, price, freeze_days) VALUES
  (1, 'Разовое посещение',            1,   1,     800.00,  0),
  (2, 'Месяц, 8 посещений',          30,   8,    3200.00,  0),
  (3, 'Месяц, безлимит',             30,   NULL, 4500.00,  0),
  (4, 'Три месяца, безлимит',        90,   NULL, 11500.00, 14),
  (5, 'Полгода, безлимит',          180,   NULL, 19900.00, 30),
  (6, 'Год, безлимит',              365,   NULL, 32000.00, 60),
  (7, 'Дневной, месяц (до 17:00)',   30,   NULL, 2900.00,  0);
```

Пример заполнения зависимой таблицы абонементов. В каждой строке указаны коды клиента и тарифа, которые уже есть в родительских таблицах:

```
INSERT INTO membership
  (membership_id, client_id, type_id, card_number, start_date, end_date,
   price_paid, status) VALUES
  (1,  1, 6, 'FC000001', '2026-01-10', '2027-01-10', 32000.00, 'активен'),
  (2,  2, 3, 'FC000002', '2026-09-20', '2026-10-20',  4500.00, 'активен'),
  (3,  3, 5, 'FC000003', '2026-05-01', '2026-10-28', 19900.00, 'активен'),
  (4,  4, 2, 'FC000004', '2026-09-25', '2026-10-25',  3200.00, 'активен'),
```

```
(5,  5, 4, 'FC000005', '2026-07-15', '2026-10-13', 11500.00, 'активен'),
(6,  6, 3, 'FC000006', '2026-08-20', '2026-09-19',  4500.00, 'истёк'),
(7,  7, 6, 'FC000007', '2025-12-01', '2026-12-01', 30400.00, 'активен'),
(8,  8, 7, 'FC000008', '2026-09-28', '2026-10-28',  2900.00, 'активен'),
(9,  9, 4, 'FC000009', '2026-08-01', '2026-11-13', 11500.00, 'заморожен'),
(10, 6, 2, 'FC000010', '2026-10-01', '2026-10-31',  3200.00, 'активен');
```

Число записей в каждой таблице после выполнения скрипта приведено в таблице 14.

Таблица 14 — Количество записей в таблицах базы данных

Таблица	Содержание	Записей
membership_type	Тарифы	7
client	Клиенты	10
membership	Абонементы	10
trainer	Тренеры	7
hall	Залы	6
equipment	Оборудование	8
workout	Виды тренировок	7
schedule	Расписание	10
booking	Записи на занятия	20
visit	Посещения	18

По данным таблицы в каждой таблице базы не меньше шести записей, что соответствует требованиям задания. Всего в базу внесено 103 записи.

2.3 SQL-запросы. Команда SELECT

В разделе приведены семь запросов на выборку данных. Для каждого запроса даны описание, текст команды и результат её выполнения на тестовых данных. Отчётной датой во всех запросах считается 7 октября 2026 года. Приёмы построения запросов предварительно отрабатывались на учебных задачах [12].

По методическим рекомендациям результат каждого запроса приводится скриншотом экрана. Таблицы ниже содержат настоящие результаты выполнения этих команд в MySQL 8.0. Выполните скрипты у себя в MySQL Workbench и замените таблицы результатов своими скриншотами: значения в них должны совпасть.

Запрос а. Соединение таблиц по равенству полей в условии WHERE. Требуется получить список действующих абонементов: фамилию и имя клиента, номер клубной карты, название тарифа и дату окончания. Данные лежат в трёх таблицах, поэтому они перечисляются в разделе FROM через запятую, а связи между ними задаются равенствами в условии WHERE. Строки упорядочены по дате окончания, чтобы первыми шли абонементы, которые закончатся раньше.

```
SELECT c.last_name   AS Фамилия,
       c.first_name  AS Имя,
       m.card_number AS Карта,
       t.type_name   AS Тариф,
       m.end_date    AS Действует_до
FROM client c, membership m, membership_type t
WHERE c.client_id = m.client_id
  AND m.type_id   = t.type_id
  AND m.status    = 'активен'
ORDER BY m.end_date;
```

Результат выполнения запроса показан в таблице 15.

Таблица 15 — Результат выполнения запроса а

Фамилия	Имя	Карта	Тариф	Действует_до
Тихонов	Павел	FC000005	Три месяца, безлимит	2026-10-13
Кузнецова	Мария	FC000002	Месяц, безлимит	2026-10-20
Белова	Анна	FC000004	Месяц, 8 посещений	2026-10-25
Орлов	Дмитрий	FC000003	Полгода, безлимит	2026-10-28
Мельникова	Ольга	FC000008	Дневной, месяц (до 17:00)	2026-10-28
Громова	Екатерина	FC000010	Месяц, 8 посещений	2026-10-31
Зайцев	Илья	FC000007	Год, безлимит	2026-12-01
Соколов	Артём	FC000001	Год, безлимит	2027-01-10

Запрос вернул 8 действующих абонементов. Раньше всех, 13 октября 2026 года, заканчивается абонемент Тихонова Павла, позже всех, в январе 2027 года, годовой абонемент Соколова Артёма.

Запрос б. Тот же запрос с использованием INNER JOIN. Запрос решает ту же задачу, но связи между таблицами записаны явно, через INNER JOIN ... ON. Условие WHERE при этом содержит только отбор по статусу. Такая запись нагляднее: условия соединения отделены от условий отбора, и случайно пропустить связь между таблицами невозможно.

```
SELECT c.last_name   AS Фамилия,
       c.first_name  AS Имя,
       m.card_number AS Карта,
       t.type_name   AS Тариф,
       m.end_date    AS Действует_до
FROM client c
  INNER JOIN membership m      ON m.client_id = c.client_id
  INNER JOIN membership_type t ON t.type_id   = m.type_id
WHERE m.status = 'активен'
ORDER BY m.end_date;
```

Результат выполнения запроса показан в таблице 16.

Таблица 16 — Результат выполнения запроса б

Фамилия	Имя	Карта	Тариф	Действует_до
Тихонов	Павел	FC000005	Три месяца, безлимит	2026-10-13
Кузнецова	Мария	FC000002	Месяц, безлимит	2026-10-20
Белова	Анна	FC000004	Месяц, 8 посещений	2026-10-25

Фамилия	Имя	Карта	Тариф	Действует_до
Орлов	Дмитрий	FC000003	Полгода, безлимит	2026-10-28
Мельникова	Ольга	FC000008	Дневной, месяц (до 17:00)	2026-10-28
Громова	Екатерина	FC000010	Месяц, 8 посещений	2026-10-31
Зайцев	Илья	FC000007	Год, безлимит	2026-12-01
Соколов	Артём	FC000001	Год, безлимит	2027-01-10

Результат полностью совпадает с результатом запроса а: те же 8 строк в том же порядке. Оба способа соединения равнозначны, дальше в работе используется запись через JOIN.

Запрос в. Использование процедурных возможностей SQL (команда CASE). Администратору нужно видеть, кому из клиентов пора предложить продление. Запрос для каждого абонемента считает число дней до окончания на отчётную дату 7 октября 2026 года и с помощью выражения CASE относит абонемент к одной из групп: заморожен, срок истёк, истекает в течение недели, истекает в течение месяца, действует.

```
SELECT m.card_number AS Карта,
       CONCAT(c.last_name, ' ', c.first_name) AS Клиент,
       m.end_date AS Действует_до,
       DATEDIFF(m.end_date, '2026-10-07') AS Осталось_дней,
       CASE
         WHEN m.status = 'заморожен' THEN 'Заморожен'
         WHEN m.end_date < '2026-10-07' THEN 'Срок истёк'
         WHEN DATEDIFF(m.end_date, '2026-10-07') <= 7
              THEN 'Истекает в течение недели'
         WHEN DATEDIFF(m.end_date, '2026-10-07') <= 30
              THEN 'Истекает в течение месяца'
         ELSE 'Действует'
       END AS Состояние
FROM membership m
  INNER JOIN client c ON c.client_id = m.client_id
ORDER BY m.end_date;
```

Результат выполнения запроса показан в таблице 17.

Таблица 17 — Результат выполнения запроса в

Карта	Клиент	Действует_до	Осталось_дней	Состояние
FC000006	Громова Екатерина	2026-09-19	-18	Срок истёк
FC000005	Тихонов Павел	2026-10-13	6	Истекает в течение недели
FC000002	Кузнецова Мария	2026-10-20	13	Истекает в течение месяца
FC000004	Белова Анна	2026-10-25	18	Истекает в течение месяца
FC000003	Орлов Дмитрий	2026-10-28	21	Истекает в течение месяца
FC000008	Мельникова Ольга	2026-10-28	21	Истекает в течение месяца

Карта	Клиент	Действует_до	Осталось_дней	Состояние
FC000010	Громова Екатерина	2026-10-31	24	Истекает в течение месяца
FC000009	Фролов Никита	2026-11-13	37	Заморожен
FC000007	Зайцев Илья	2026-12-01	55	Действует
FC000001	Соколов Артём	2027-01-10	95	Действует

Из 10 абонементов один уже истёк, один заканчивается в течение недели, пять заканчиваются в течение месяца, один заморожен и два действуют дольше месяца. Клиентам второй и третьей группы стоит предложить продление.

Запрос г. Группировка, групповые функции и условие на группы (HAVING). Запрос показывает загрузку тренеров за неделю с 5 по 11 октября 2026 года: сколько занятий стоит в расписании у каждого и сколько клиентов на них записано (отменённые записи не считаются). Строки группируются по тренеру, а условие HAVING оставляет только тех, у кого не меньше двух занятий.

```
SELECT CONCAT(t.last_name, ' ', t.first_name) AS Тренер,
       COUNT(DISTINCT s.schedule_id) AS Занятий,
       COUNT(b.booking_id)           AS Записей,
       ROUND(COUNT(b.booking_id) /
             COUNT(DISTINCT s.schedule_id), 1) AS В_среднем_на_занятие
FROM trainer t
  INNER JOIN schedule s ON s.trainer_id = t.trainer_id
  LEFT JOIN booking b   ON b.schedule_id = s.schedule_id
                       AND b.status <> 'отменил'
WHERE s.start_time >= '2026-10-05'
  AND s.start_time <  '2026-10-12'
GROUP BY t.trainer_id, t.last_name, t.first_name
HAVING COUNT(DISTINCT s.schedule_id) >= 2
ORDER BY Записей DESC;
```

Результат выполнения запроса показан в таблице 18.

Таблица 18 — Результат выполнения запроса г

Тренер	Занятий	Записей	В_среднем_на_занятие
Лебедева Ирина	3	6	2.0
Крылов Максим	2	5	2.5
Назаров Тимур	2	4	2.0

Условию удовлетворяют 3 тренера из семи. Больше всего занятий у Лебедевой Ирины (три), а средняя наполняемость выше у Крылова Максима. Тренеры с одним занятием и тренер без занятий в результат не попали.

Запрос д. Левое внешнее соединение (LEFT JOIN). Запрос строит сводку по всем клиентам: сколько абонементов куплено, сколько раз клиент приходил в клуб и когда был последний раз. Левое соединение нужно для того,

чтобы в отчёт попали и клиенты, у которых ещё нет ни абонемента, ни посещений: при внутреннем соединении они бы пропали.

```
SELECT CONCAT(c.last_name, ' ', c.first_name) AS Клиент,
       COUNT(DISTINCT m.membership_id) AS Абонементов,
       COUNT(v.visit_id)               AS Посещений,
       MAX(DATE(v.entry_time))         AS Последний_визит
FROM client c
  LEFT JOIN membership m ON m.client_id = c.client_id
  LEFT JOIN visit v      ON v.membership_id = m.membership_id
GROUP BY c.client_id, c.last_name, c.first_name
ORDER BY Посещений DESC, Клиент;
```

Результат выполнения запроса показан в таблице 19.

Таблица 19 — Результат выполнения запроса д

Клиент	Абонементов	Посещений	Последний_визит
Орлов Дмитрий	1	3	2026-10-07
Соколов Артём	1	3	2026-10-06
Белова Анна	1	2	2026-10-07
Громова Екатерина	2	2	2026-09-17
Зайцев Илья	1	2	2026-10-06
Кузнецова Мария	1	2	2026-10-05
Тихонов Павел	1	2	2026-10-06
Мельникова Ольга	1	1	2026-10-07
Фролов Никита	1	1	2026-08-05
Жукова Дарья	0	0	—

В результате все 10 клиентов. У Жуковой Дарьи, которая только зарегистрировалась, нет ни абонемента, ни посещений: в столбце с датой стоит NULL. Чаще всех в клуб ходят Орлов Дмитрий и Соколов Артём.

Запрос е. Вложенный подзапрос. Запрос находит абонементы, за которые заплачено больше, чем в среднем по клубу. Средняя сумма заранее неизвестна, поэтому она вычисляется вложенным запросом SELECT AVG(...), а внешний запрос сравнивает с ней каждую строку.

```
SELECT m.card_number AS Карта,
       CONCAT(c.last_name, ' ', c.first_name) AS Клиент,
       t.type_name  AS Тариф,
       m.price_paid AS Оплачено
FROM membership m
  INNER JOIN client c          ON c.client_id = m.client_id
  INNER JOIN membership_type t ON t.type_id   = m.type_id
WHERE m.price_paid > (SELECT AVG(price_paid) FROM membership)
ORDER BY m.price_paid DESC;
```

Результат выполнения запроса показан в таблице 20.

Таблица 20 — Результат выполнения запроса е

Карта	Клиент	Тариф	Оплачено
FC000001	Соколов Артём	Год, безлимит	32000.00

Карта	Клиент	Тариф	Оплачено
FC000007	Зайцев Илья	Год, безлимит	30400.00
FC000003	Орлов Дмитрий	Полгода, безлимит	19900.00

Средняя стоимость проданного абонемента составляет 12 360 рублей. Выше неё оказались 3 абонемента: два годовых и один на полгода. Именно длинные абонементы приносят клубу основную выручку.

Запрос ж. Создание представления (VIEW). Список действующих абонементов нужен администратору постоянно, поэтому запрос б удобно сохранить как представление. В представление дополнительно включён телефон клиента, чтобы по нему можно было звонить с предложением продления.

```
CREATE OR REPLACE VIEW v_active_membership AS
SELECT c.client_id,
       c.last_name,
       c.first_name,
       c.phone,
       m.card_number,
       t.type_name,
       m.end_date
FROM client c
  INNER JOIN membership m      ON m.client_id = c.client_id
  INNER JOIN membership_type t ON t.type_id   = m.type_id
WHERE m.status = 'активен';
```

После этого к представлению можно обращаться как к обычной таблице. Например, так выглядит список клиентов, у которых абонемент заканчивается до конца октября:

```
SELECT last_name   AS Фамилия,
       first_name  AS Имя,
       phone       AS Телефон,
       type_name   AS Тариф,
       end_date    AS Действует_до
FROM v_active_membership
WHERE end_date <= '2026-10-31'
ORDER BY end_date;
```

Результат обращения к представлению показан в таблице 21.

Таблица 21 — Результат выборки из представления v_active_membership

Фамилия	Имя	Телефон	Тариф	Действует_до
Тихонов	Павел	+79161110105	Три месяца, безлимит	2026-10-13
Кузнецова	Мария	+79161110102	Месяц, безлимит	2026-10-20
Белова	Анна	+79161110104	Месяц, 8 посещений	2026-10-25
Орлов	Дмитрий	+79161110103	Полгода, безлимит	2026-10-28
Мельникова	Ольга	+79161110108	Дневной, месяц (до 17:00)	2026-10-28
Громова	Екатерина	+79161110106	Месяц, 8 посещений	2026-10-31

Запрос вернул 6 строк. Обращение к представлению занимает несколько строк вместо соединения трёх таблиц, а условие отбора активных абонементов записано один раз и не может быть забыто.

2.4 Триггеры и хранимые процедуры

Часть бизнес-правил нельзя выразить ограничениями таблиц, потому что для проверки нужно обратиться к другим строкам или другим таблицам. Такие правила реализованы триггерами — процедурами, которые СУБД сама запускает при изменении данных. Действия, которые выполняет пользователь, оформлены хранимыми процедурами. Всего создано три триггера и две процедуры, их полный текст приведён в приложении В.

Триггер trg_membership_bi срабатывает перед добавлением абонемента. Если дата окончания или сумма не указаны, он берёт срок и цену из тарифа и рассчитывает их сам. Администратору достаточно выбрать клиента, тариф и дату начала.

```
CREATE TRIGGER trg_membership_bi
BEFORE INSERT ON membership
FOR EACH ROW
BEGIN
  DECLARE v_days  SMALLINT UNSIGNED;
  DECLARE v_price DECIMAL(10,2);

  SELECT duration_days, price INTO v_days, v_price
  FROM membership_type
  WHERE type_id = NEW.type_id;

  IF NEW.end_date IS NULL THEN
    SET NEW.end_date = DATE_ADD(NEW.start_date, INTERVAL v_days DAY);
  END IF;
  IF NEW.price_paid IS NULL THEN
    SET NEW.price_paid = v_price;
  END IF;
END;
```

Проверка: клиенту с кодом 10 продаётся месячный безлимитный абонемент, дата окончания и сумма не указываются.

```
INSERT INTO membership (client_id, type_id, card_number, start_date)
VALUES (10, 3, 'FC000011', '2026-10-07');

SELECT card_number AS Карта, start_date AS Начало, end_date AS Окончание,
       price_paid AS Оплачено, status AS Статус
FROM membership
WHERE card_number = 'FC000011';
```

Результат показан в таблице 22.

Таблица 22 — Абонемент, добавленный с участием триггера

Карта	Начало	Окончание	Оплачено	Статус
FC000011	2026-10-07	2026-11-06	4500.00	активен

Триггер подставил дату окончания 6 ноября 2026 года (30 дней от даты начала) и цену тарифа 4 500 рублей.

Триггер trg_visit_bi срабатывает перед регистрацией посещения. Он проверяет, что абонемент активен, что дата входа попадает в срок действия и что лимит посещений не исчерпан. При нарушении командой SIGNAL формируется ошибка, и строка не добавляется. Текст этого и следующего триггера приведён в приложении В.

Проверка: попытка пройти в клуб по абонементу с кодом 6, срок которого истёк 19 сентября.

```
INSERT INTO visit (membership_id, entry_time)
VALUES (6, '2026-10-07 18:30:00');
```

СУБД отклонила команду и вернула сообщение: «ERROR 1644 (45000): Проход запрещён: абонемент не действует». Проход по только что проданному абонементу FC000011 выполняется без ошибок.

Триггер trg_booking_bi срабатывает перед записью на занятие. Он считает неотменённые записи на это занятие и сравнивает их с числом мест, а затем проверяет, что у клиента на дату занятия есть активный абонемент.

Проверка состоит из трёх попыток: записаться на персональную тренировку, где единственное место занято; записать клиента с замороженным абонементом; записать клиента с действующим абонементом на занятие, где места есть.

```
INSERT INTO booking (schedule_id, client_id, booked_at)
VALUES (6, 1, '2026-10-07 12:00:00');

INSERT INTO booking (schedule_id, client_id, booked_at)
VALUES (9, 9, '2026-10-07 12:05:00');

INSERT INTO booking (schedule_id, client_id, booked_at)
VALUES (9, 10, '2026-10-07 12:10:00');
```

Первая команда отклонена с сообщением «Запись невозможна: свободных мест нет», вторая — с сообщением «Запись невозможна: нет действующего абонемента». Третья команда выполнена, и на занятии стало три записи.

Процедура sp_freeze_membership замораживает абонемент на заданное число дней: меняет статус и сдвигает дату окончания. Перед этим она проверяет, что абонемент существует, активен и что срок заморозки не превышает допустимый по тарифу. Текст процедуры приведён в приложении В.

Проверка: попытка заморозить на 10 дней месячный абонемент, у которого заморозка не предусмотрена, и заморозка на 14 дней полугодового абонемента.

```
CALL sp_freeze_membership(2, 10);
CALL sp_freeze_membership(3, 14);
```

```
SELECT card_number AS Карта, end_date AS Окончание, status AS Статус
FROM membership
WHERE membership_id = 3;
```

Первый вызов завершился ошибкой «Срок заморозки превышает допустимый по тарифу». Результат второго вызова показан в таблице 23.

Таблица 23 — Абонемент после вызова процедуры заморозки

Карта	Окончание	Статус
FC000003	2026-11-11	заморожен

Дата окончания абонемента сдвинулась с 28 октября на 11 ноября 2026 года, статус изменился на «заморожен».

Процедура sp_close_expired переводит в статус «истёк» все активные абонементы, срок которых закончился раньше указанной даты, и возвращает через выходной параметр число изменённых строк. Её удобно запускать раз в сутки.

```
CREATE PROCEDURE sp_close_expired (
  IN  p_date  DATE,
  OUT p_count INT
)
BEGIN
  UPDATE membership
  SET status = 'истёк'
  WHERE end_date < p_date
    AND status = 'активен';

  SET p_count = ROW_COUNT();
END;
```

Проверка: закрытие абонементов по состоянию на 15 октября 2026 года.

```
CALL sp_close_expired('2026-10-15', @closed);
SELECT @closed AS Закрыто_абонементов;
```

Процедура вернула значение 1: закрыт абонемент FC000005, срок которого закончился 13 октября. Состояние просроченных абонементов после вызова показано в таблице 24.

Таблица 24 — Абонементы со сроком окончания до 15 октября 2026 года

Карта	Окончание	Статус
FC000006	2026-09-19	истёк
FC000005	2026-10-13	истёк

Оба абонемента с истёкшим сроком имеют статус «истёк». Абонемент FC000006 был закрыт раньше, поэтому процедура его не изменяла и в счётчик он не попал.

ЗАКЛЮЧЕНИЕ

В работе спроектирована и реализована база данных фитнес-клуба. Цель работы достигнута, все поставленные задачи решены.

На этапе проектирования описана предметная область и сформулированы десять бизнес-правил. Построена концептуальная модель из десяти сущностей и девяти связей, определены домены атрибутов. Модель преобразована в даталогическую реляционную модель, таблицы которой находятся в третьей нормальной форме.

На этапе реализации написан сценарий создания базы данных в СУБД MySQL 8.0: десять таблиц с первичными и внешними ключами, ограничениями уникальности и проверочными ограничениями, четыре индекса и представление. Таблицы заполнены согласованными тестовыми данными, всего 103 записи. Составлены и выполнены семь запросов: с соединением через условие WHERE и через INNER JOIN, с выражением CASE, с группировкой и условием HAVING, с левым соединением, с вложенным подзапросом и запрос на создание представления.

Бизнес-правила, которые нельзя задать ограничениями таблиц, реализованы тремя триггерами: автоматический расчёт срока и стоимости абонемента, запрет прохода по недействующему абонементу, контроль числа мест на занятии. Заморозка абонемента и закрытие просроченных абонементов оформлены хранимыми процедурами. Все объекты проверены на тестовых данных, включая попытки нарушить правила: СУБД отклоняет такие операции с понятным сообщением.

Основной вывод работы: значительную часть логики учёта можно и нужно закреплять в самой базе данных. Ограничения и триггеры защищают данные независимо от того, из какой программы к ним обращаются.

Работу можно развивать в нескольких направлениях:

– добавить учёт платежей отдельной таблицей, чтобы поддержать оплату в рассрочку и возвраты;

– хранить историю заморозок, а не только итоговую дату окончания;

– ввести роли пользователей и разграничить права доступа к персональным данным клиентов;

– добавить поддержку нескольких филиалов клуба, для чего потребуется перейти к распределённой базе данных [3];

– настроить планировщик событий MySQL для ежедневного вызова процедуры закрытия абонементов;

– разработать клиентское приложение для администратора и личный кабинет клиента.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. О персональных данных: федеральный закон от 27.07.2006 № 152-ФЗ.

2. ГОСТ 20886-85. Организация данных в системах обработки данных. Термины и определения. — М.: Изд-во стандартов, 1985. — 8 с.

3. Агальцов В. П. Базы данных: в 2 кн. Кн. 2. Распределенные и удаленные базы данных: учебник. — М.: ИД «ФОРУМ»: ИНФРА-М, 2020. — 271 с.

4. Голицына О. Л., Максимов Н. В., Попов И. И. Базы данных: учеб. пособие. — 4-е изд., перераб. и доп. — М.: ФОРУМ: ИНФРА-М, 2018. — 400 с.

5. Нестеров С. А. Базы данных: учебник и практикум для вузов. — М.: Юрайт, 2023.

6. Новиков Б. А., Горшкова Е. А., Графеева Н. Г. Основы технологий баз данных: учеб. пособие. — М.: ДМК Пресс, 2020.

7. Стружкин Н. П., Годин В. В. Базы данных: проектирование: учебник для вузов. — М.: Юрайт, 2023.

8. Шустова Л. И., Тараканов О. В. Базы данных: учебник. — М.: ИНФРА-М, 2019. — 304 с.

9. Карпова Т. Базы данных: модели, разработка, реализация [Электронный ресурс] // Национальный Открытый Университет «ИНТУИТ». — URL: http://www.intuit.ru/studies/courses/1001/297/info (дата обращения: 07.10.2026).

10. Кузнецов С. Д. Базы данных. Вводный курс [Электронный ресурс]. — URL: http://citforum.ru/database/advanced_intro (дата обращения: 07.10.2026).

11. Назаров А. Введение в СУБД MySQL [Электронный ресурс] // Национальный Открытый Университет «ИНТУИТ». — URL: http://www.intuit.ru/studies/courses/111/111/info (дата обращения: 07.10.2026).

12. Практическое владение языком SQL. Упражнения по SQL [Электронный ресурс]. — URL: http://sql-ex.ru (дата обращения: 07.10.2026).

13. Туманов В. Основы проектирования реляционных баз данных [Электронный ресурс] // Национальный Открытый Университет «ИНТУИТ». — URL: http://www.intuit.ru/studies/courses/1095/191/info (дата обращения: 07.10.2026).

14. DB-Engines Ranking [Электронный ресурс]. — URL: https://db-engines.com/en/ranking (дата обращения: 07.10.2026).

15. MySQL 8.0 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/refman/8.0/en/ (дата обращения: 07.10.2026).

16. MySQL Workbench Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/workbench/en/ (дата обращения: 07.10.2026).

Приложение А

Сценарий создания объектов базы данных

```
-- База данных фитнес-клуба. Сценарий создания объектов (MySQL 8.0)
DROP DATABASE IF EXISTS fitness_club;
CREATE DATABASE fitness_club
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE fitness_club;

-- Типы абонементов (тарифы)
CREATE TABLE membership_type (
  type_id       INT UNSIGNED      NOT NULL AUTO_INCREMENT,
  type_name     VARCHAR(60)       NOT NULL,
  duration_days SMALLINT UNSIGNED NOT NULL,
  visit_limit   SMALLINT UNSIGNED NULL,
  price         DECIMAL(10,2)     NOT NULL,
  freeze_days   TINYINT UNSIGNED  NOT NULL DEFAULT 0,
  CONSTRAINT pk_membership_type PRIMARY KEY (type_id),
  CONSTRAINT uq_type_name UNIQUE (type_name),
  CONSTRAINT chk_type_price CHECK (price >= 0),
  CONSTRAINT chk_type_duration CHECK (duration_days > 0)
) ENGINE = InnoDB;

-- Клиенты
CREATE TABLE client (
  client_id   INT UNSIGNED   NOT NULL AUTO_INCREMENT,
  last_name   VARCHAR(50)    NOT NULL,
  first_name  VARCHAR(50)    NOT NULL,
  middle_name VARCHAR(50)    NULL,
  birth_date  DATE           NOT NULL,
  gender      ENUM('М','Ж')  NOT NULL,
  phone       CHAR(12)       NOT NULL,
  email       VARCHAR(100)   NULL,
  reg_date    DATE           NOT NULL,
  CONSTRAINT pk_client PRIMARY KEY (client_id),
  CONSTRAINT uq_client_phone UNIQUE (phone),
  CONSTRAINT uq_client_email UNIQUE (email)
) ENGINE = InnoDB;

-- Абонементы (клубные карты), проданные клиентам
CREATE TABLE membership (
  membership_id INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  client_id     INT UNSIGNED  NOT NULL,
  type_id       INT UNSIGNED  NOT NULL,
  card_number   CHAR(8)       NOT NULL,
  start_date    DATE          NOT NULL,
  end_date      DATE          NOT NULL,
  price_paid    DECIMAL(10,2) NOT NULL,
  status        ENUM('активен','заморожен','истёк')
                              NOT NULL DEFAULT 'активен',
  CONSTRAINT pk_membership PRIMARY KEY (membership_id),
  CONSTRAINT uq_membership_card UNIQUE (card_number),
  CONSTRAINT fk_membership_client FOREIGN KEY (client_id)
    REFERENCES client (client_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_membership_type FOREIGN KEY (type_id)
    REFERENCES membership_type (type_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_membership_dates CHECK (end_date >= start_date),
```

```
  CONSTRAINT chk_membership_price CHECK (price_paid >= 0)
) ENGINE = InnoDB;

-- Тренеры
CREATE TABLE trainer (
  trainer_id     INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  last_name      VARCHAR(50)   NOT NULL,
  first_name     VARCHAR(50)   NOT NULL,
  middle_name    VARCHAR(50)   NULL,
  specialization VARCHAR(80)   NOT NULL,
  phone          CHAR(12)      NOT NULL,
  hire_date      DATE          NOT NULL,
  hour_rate      DECIMAL(8,2)  NOT NULL,
  CONSTRAINT pk_trainer PRIMARY KEY (trainer_id),
  CONSTRAINT uq_trainer_phone UNIQUE (phone),
  CONSTRAINT chk_trainer_rate CHECK (hour_rate > 0)
) ENGINE = InnoDB;

-- Залы
CREATE TABLE hall (
  hall_id   INT UNSIGNED      NOT NULL AUTO_INCREMENT,
  hall_name VARCHAR(60)       NOT NULL,
  capacity  SMALLINT UNSIGNED NOT NULL,
  area      DECIMAL(6,1)      NOT NULL,
  CONSTRAINT pk_hall PRIMARY KEY (hall_id),
  CONSTRAINT uq_hall_name UNIQUE (hall_name)
) ENGINE = InnoDB;

-- Оборудование, закреплённое за залами
CREATE TABLE equipment (
  equipment_id   INT UNSIGNED NOT NULL AUTO_INCREMENT,
  hall_id        INT UNSIGNED NOT NULL,
  equipment_name VARCHAR(100) NOT NULL,
  inventory_no   VARCHAR(10)  NOT NULL,
  purchase_date  DATE         NOT NULL,
  state          ENUM('исправно','в ремонте','списано')
                              NOT NULL DEFAULT 'исправно',
  CONSTRAINT pk_equipment PRIMARY KEY (equipment_id),
  CONSTRAINT uq_equipment_inv UNIQUE (inventory_no),
  CONSTRAINT fk_equipment_hall FOREIGN KEY (hall_id)
    REFERENCES hall (hall_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE = InnoDB;

-- Виды тренировок
CREATE TABLE workout (
  workout_id   INT UNSIGNED      NOT NULL AUTO_INCREMENT,
  workout_name VARCHAR(80)       NOT NULL,
  workout_kind ENUM('групповая','персональная') NOT NULL,
  duration_min SMALLINT UNSIGNED NOT NULL,
  extra_price  DECIMAL(8,2)      NOT NULL DEFAULT 0,
  CONSTRAINT pk_workout PRIMARY KEY (workout_id),
  CONSTRAINT uq_workout_name UNIQUE (workout_name),
  CONSTRAINT chk_workout_duration CHECK (duration_min BETWEEN 15 AND 240)
) ENGINE = InnoDB;

-- Расписание занятий
CREATE TABLE schedule (
  schedule_id INT UNSIGNED      NOT NULL AUTO_INCREMENT,
  workout_id  INT UNSIGNED      NOT NULL,
```

```
  trainer_id  INT UNSIGNED      NOT NULL,
  hall_id     INT UNSIGNED      NOT NULL,
  start_time  DATETIME          NOT NULL,
  max_places  SMALLINT UNSIGNED NOT NULL,
  CONSTRAINT pk_schedule PRIMARY KEY (schedule_id),
  CONSTRAINT uq_schedule_hall UNIQUE (hall_id, start_time),
  CONSTRAINT uq_schedule_trainer UNIQUE (trainer_id, start_time),
  CONSTRAINT fk_schedule_workout FOREIGN KEY (workout_id)
    REFERENCES workout (workout_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_schedule_trainer FOREIGN KEY (trainer_id)
    REFERENCES trainer (trainer_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_schedule_hall FOREIGN KEY (hall_id)
    REFERENCES hall (hall_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_schedule_places CHECK (max_places > 0)
) ENGINE = InnoDB;

-- Записи клиентов на занятия
CREATE TABLE booking (
  booking_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  schedule_id INT UNSIGNED NOT NULL,
  client_id   INT UNSIGNED NOT NULL,
  booked_at   DATETIME     NOT NULL,
  status      ENUM('записан','посетил','отменил')
                           NOT NULL DEFAULT 'записан',
  CONSTRAINT pk_booking PRIMARY KEY (booking_id),
  CONSTRAINT uq_booking UNIQUE (schedule_id, client_id),
  CONSTRAINT fk_booking_schedule FOREIGN KEY (schedule_id)
    REFERENCES schedule (schedule_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_booking_client FOREIGN KEY (client_id)
    REFERENCES client (client_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE = InnoDB;

-- Посещения клуба (проход по клубной карте)
CREATE TABLE visit (
  visit_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  membership_id INT UNSIGNED NOT NULL,
  entry_time    DATETIME     NOT NULL,
  exit_time     DATETIME     NULL,
  CONSTRAINT pk_visit PRIMARY KEY (visit_id),
  CONSTRAINT fk_visit_membership FOREIGN KEY (membership_id)
    REFERENCES membership (membership_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_visit_times
    CHECK (exit_time IS NULL OR exit_time > entry_time)
) ENGINE = InnoDB;

-- Индексы для частых условий поиска
CREATE INDEX idx_client_name     ON client (last_name, first_name);
CREATE INDEX idx_membership_end  ON membership (end_date, status);
CREATE INDEX idx_schedule_start  ON schedule (start_time);
CREATE INDEX idx_visit_entry     ON visit (entry_time);

-- Представление: расписание в читаемом виде
CREATE VIEW v_schedule_full AS
SELECT s.schedule_id,
```

```
       s.start_time,
       w.workout_name,
       w.workout_kind,
       CONCAT(t.last_name, ' ', t.first_name) AS trainer_name,
       h.hall_name,
       s.max_places
FROM schedule s
  INNER JOIN workout w ON w.workout_id = s.workout_id
  INNER JOIN trainer t ON t.trainer_id = s.trainer_id
  INNER JOIN hall    h ON h.hall_id    = s.hall_id;
```

Приложение Б

Скрипт заполнения таблиц тестовыми данными

```
-- Заполнение таблиц тестовыми данными
USE fitness_club;

INSERT INTO membership_type
  (type_id, type_name, duration_days, visit_limit, price, freeze_days) VALUES
  (1, 'Разовое посещение',            1,   1,     800.00,  0),
  (2, 'Месяц, 8 посещений',          30,   8,    3200.00,  0),
  (3, 'Месяц, безлимит',             30,   NULL, 4500.00,  0),
  (4, 'Три месяца, безлимит',        90,   NULL, 11500.00, 14),
  (5, 'Полгода, безлимит',          180,   NULL, 19900.00, 30),
  (6, 'Год, безлимит',              365,   NULL, 32000.00, 60),
  (7, 'Дневной, месяц (до 17:00)',   30,   NULL, 2900.00,  0);

INSERT INTO client
  (client_id, last_name, first_name, middle_name, birth_date, gender,
   phone, email, reg_date) VALUES
  (1,  'Соколов',    'Артём',     'Игоревич',   '1998-03-14', 'М',
       '+79161110101', 'sokolov.a@example.ru',   '2025-11-02'),
  (2,  'Кузнецова',  'Мария',     'Андреевна',  '2001-07-22', 'Ж',
       '+79161110102', 'kuznetsova.m@example.ru','2026-09-20'),
  (3,  'Орлов',      'Дмитрий',   'Сергеевич',  '1989-12-05', 'М',
       '+79161110103', 'orlov.d@example.ru',     '2026-04-28'),
  (4,  'Белова',     'Анна',      'Викторовна', '1995-05-30', 'Ж',
       '+79161110104', 'belova.a@example.ru',    '2026-09-25'),
  (5,  'Тихонов',    'Павел',     'Олегович',   '2003-09-11', 'М',
       '+79161110105', 'tikhonov.p@example.ru',  '2026-07-15'),
  (6,  'Громова',    'Екатерина', 'Юрьевна',    '1992-01-19', 'Ж',
       '+79161110106', 'gromova.e@example.ru',   '2026-08-20'),
  (7,  'Зайцев',     'Илья',      'Романович',  '1985-08-08', 'М',
       '+79161110107', 'zaytsev.i@example.ru',   '2025-12-01'),
  (8,  'Мельникова', 'Ольга',     'Павловна',   '1978-04-27', 'Ж',
       '+79161110108', NULL,                     '2026-09-28'),
  (9,  'Фролов',     'Никита',    'Алексеевич', '2005-02-16', 'М',
       '+79161110109', 'frolov.n@example.ru',    '2026-08-01'),
  (10, 'Жукова',     'Дарья',     'Максимовна', '1999-10-03', 'Ж',
       '+79161110110', 'zhukova.d@example.ru',   '2026-10-06');

INSERT INTO membership
  (membership_id, client_id, type_id, card_number, start_date, end_date,
   price_paid, status) VALUES
  (1,  1, 6, 'FC000001', '2026-01-10', '2027-01-10', 32000.00, 'активен'),
  (2,  2, 3, 'FC000002', '2026-09-20', '2026-10-20',  4500.00, 'активен'),
  (3,  3, 5, 'FC000003', '2026-05-01', '2026-10-28', 19900.00, 'активен'),
  (4,  4, 2, 'FC000004', '2026-09-25', '2026-10-25',  3200.00, 'активен'),
  (5,  5, 4, 'FC000005', '2026-07-15', '2026-10-13', 11500.00, 'активен'),
  (6,  6, 3, 'FC000006', '2026-08-20', '2026-09-19',  4500.00, 'истёк'),
  (7,  7, 6, 'FC000007', '2025-12-01', '2026-12-01', 30400.00, 'активен'),
  (8,  8, 7, 'FC000008', '2026-09-28', '2026-10-28',  2900.00, 'активен'),
  (9,  9, 4, 'FC000009', '2026-08-01', '2026-11-13', 11500.00, 'заморожен'),
  (10, 6, 2, 'FC000010', '2026-10-01', '2026-10-31',  3200.00, 'активен');

INSERT INTO trainer
  (trainer_id, last_name, first_name, middle_name, specialization,
   phone, hire_date, hour_rate) VALUES
  (1, 'Воронин',   'Алексей', 'Петрович',   'Силовой тренинг',
      '+79162220201', '2022-03-01', 1800.00),
```

```
  (2, 'Лебедева',  'Ирина',   'Сергеевна',  'Йога и пилатес',
      '+79162220202', '2023-09-15', 1500.00),
  (3, 'Назаров',   'Тимур',   'Рашидович',  'Бокс',
      '+79162220203', '2021-11-10', 1700.00),
  (4, 'Савельева', 'Юлия',    'Олеговна',   'Кардиопрограммы',
      '+79162220204', '2024-02-05', 1400.00),
  (5, 'Крылов',    'Максим',  'Денисович',  'Функциональный тренинг',
      '+79162220205', '2023-01-20', 1600.00),
  (6, 'Ершова',    'Полина',  'Игоревна',   'Стретчинг',
      '+79162220206', '2025-06-01', 1300.00),
  (7, 'Абрамов',   'Роман',   'Витальевич', 'Плавание',
      '+79162220207', '2026-09-01', 1500.00);

INSERT INTO hall (hall_id, hall_name, capacity, area) VALUES
  (1, 'Тренажёрный зал',         40, 320.0),
  (2, 'Зал групповых программ',  20, 110.5),
  (3, 'Зал йоги',                14,  72.0),
  (4, 'Кардиозона',              18,  95.0),
  (5, 'Зал единоборств',         12,  80.5),
  (6, 'Бассейн',                 16, 250.0);

INSERT INTO equipment
  (equipment_id, hall_id, equipment_name, inventory_no, purchase_date,
   state) VALUES
  (1, 4, 'Беговая дорожка',           'CD-0001', '2023-02-14', 'исправно'),
  (2, 4, 'Велотренажёр для сайкла',   'CD-0002', '2023-02-14', 'исправно'),
  (3, 4, 'Эллиптический тренажёр',    'CD-0003', '2022-08-30', 'в ремонте'),
  (4, 1, 'Силовая рама',              'TZ-0001', '2021-10-05', 'исправно'),
  (5, 1, 'Скамья для жима',           'TZ-0002', '2021-10-05', 'исправно'),
  (6, 1, 'Гантельный ряд 2-40 кг',    'TZ-0003', '2024-04-18', 'исправно'),
  (7, 5, 'Боксёрский мешок',          'ED-0001', '2020-06-22', 'списано'),
  (8, 3, 'Комплект ковриков для йоги', 'YG-0001', '2025-01-12', 'исправно');

INSERT INTO workout
  (workout_id, workout_name, workout_kind, duration_min, extra_price) VALUES
  (1, 'Йога',                   'групповая',    60,    0.00),
  (2, 'Пилатес',                'групповая',    55,    0.00),
  (3, 'Функциональный тренинг', 'групповая',    60,    0.00),
  (4, 'Сайкл',                  'групповая',    45,    0.00),
  (5, 'Бокс',                   'групповая',    90,  300.00),
  (6, 'Стретчинг',              'групповая',    50,    0.00),
  (7, 'Персональная тренировка', 'персональная', 60, 2500.00);

INSERT INTO schedule
  (schedule_id, workout_id, trainer_id, hall_id, start_time, max_places)
VALUES
  (1,  1, 2, 3, '2026-10-05 19:00:00', 12),
  (2,  3, 5, 2, '2026-10-05 20:00:00', 15),
  (3,  5, 3, 5, '2026-10-06 19:30:00', 10),
  (4,  4, 4, 4, '2026-10-06 18:00:00',  8),
  (5,  2, 2, 3, '2026-10-07 10:00:00', 12),
  (6,  7, 1, 1, '2026-10-07 18:00:00',  1),
  (7,  6, 6, 2, '2026-10-08 19:00:00', 15),
  (8,  1, 2, 3, '2026-10-08 20:00:00', 12),
  (9,  5, 3, 5, '2026-10-09 19:30:00', 10),
  (10, 3, 5, 2, '2026-10-10 12:00:00', 15);

INSERT INTO booking
  (booking_id, schedule_id, client_id, booked_at, status) VALUES
  (1,  1, 2, '2026-10-03 12:10:00', 'посетил'),
```

```
  (2,  1, 4, '2026-10-04 09:35:00', 'посетил'),
  (3,  1, 6, '2026-10-04 21:02:00', 'отменил'),
  (4,  2, 1, '2026-10-02 18:44:00', 'посетил'),
  (5,  2, 5, '2026-10-05 08:15:00', 'посетил'),
  (6,  2, 3, '2026-10-05 10:20:00', 'посетил'),
  (7,  3, 7, '2026-10-01 14:00:00', 'посетил'),
  (8,  3, 5, '2026-10-05 22:30:00', 'посетил'),
  (9,  4, 1, '2026-10-05 21:15:00', 'посетил'),
  (10, 4, 2, '2026-10-06 08:05:00', 'отменил'),
  (11, 5, 8, '2026-10-05 11:40:00', 'посетил'),
  (12, 5, 4, '2026-10-06 19:20:00', 'посетил'),
  (13, 6, 3, '2026-10-01 09:00:00', 'посетил'),
  (14, 7, 6, '2026-10-06 20:50:00', 'записан'),
  (15, 8, 2, '2026-10-06 20:11:00', 'записан'),
  (16, 8, 4, '2026-10-07 08:30:00', 'записан'),
  (17, 9, 7, '2026-10-06 21:40:00', 'записан'),
  (18, 9, 5, '2026-10-07 07:55:00', 'записан'),
  (19, 10, 1, '2026-10-06 22:05:00', 'записан'),
  (20, 10, 8, '2026-10-07 11:25:00', 'записан');

INSERT INTO visit (visit_id, membership_id, entry_time, exit_time) VALUES
  (1,  9,  '2026-08-05 18:00:00', '2026-08-05 19:20:00'),
  (2,  6,  '2026-09-10 19:00:00', '2026-09-10 20:20:00'),
  (3,  6,  '2026-09-17 19:05:00', '2026-09-17 20:30:00'),
  (4,  3,  '2026-09-29 07:30:00', '2026-09-29 08:45:00'),
  (5,  1,  '2026-10-01 18:05:00', '2026-10-01 19:40:00'),
  (6,  2,  '2026-10-02 18:30:00', '2026-10-02 19:50:00'),
  (7,  7,  '2026-10-03 11:00:00', '2026-10-03 12:30:00'),
  (8,  2,  '2026-10-05 18:40:00', '2026-10-05 20:10:00'),
  (9,  4,  '2026-10-05 18:50:00', '2026-10-05 20:05:00'),
  (10, 1,  '2026-10-05 19:45:00', '2026-10-05 21:10:00'),
  (11, 3,  '2026-10-05 19:50:00', '2026-10-05 21:15:00'),
  (12, 5,  '2026-10-05 19:55:00', '2026-10-05 21:05:00'),
  (13, 1,  '2026-10-06 17:50:00', '2026-10-06 19:05:00'),
  (14, 7,  '2026-10-06 19:15:00', '2026-10-06 21:10:00'),
  (15, 5,  '2026-10-06 19:20:00', '2026-10-06 21:05:00'),
  (16, 8,  '2026-10-07 09:45:00', '2026-10-07 11:15:00'),
  (17, 4,  '2026-10-07 09:50:00', '2026-10-07 11:10:00'),
  (18, 3,  '2026-10-07 17:50:00', NULL);
```

Приложение В

Триггеры и хранимые процедуры

```
-- Триггеры и хранимые процедуры
USE fitness_club;

DELIMITER //

-- Триггер 1. При продаже абонемента сам рассчитывает дату окончания
-- и стоимость по выбранному тарифу, если они не указаны
CREATE TRIGGER trg_membership_bi
BEFORE INSERT ON membership
FOR EACH ROW
BEGIN
  DECLARE v_days  SMALLINT UNSIGNED;
  DECLARE v_price DECIMAL(10,2);

  SELECT duration_days, price INTO v_days, v_price
  FROM membership_type
  WHERE type_id = NEW.type_id;

  IF NEW.end_date IS NULL THEN
    SET NEW.end_date = DATE_ADD(NEW.start_date, INTERVAL v_days DAY);
  END IF;
  IF NEW.price_paid IS NULL THEN
    SET NEW.price_paid = v_price;
  END IF;
END//

-- Триггер 2. Не пропускает в клуб по недействующему абонементу
-- и следит за лимитом посещений
CREATE TRIGGER trg_visit_bi
BEFORE INSERT ON visit
FOR EACH ROW
BEGIN
  DECLARE v_status VARCHAR(20);
  DECLARE v_start  DATE;
  DECLARE v_end    DATE;
  DECLARE v_limit  SMALLINT UNSIGNED;
  DECLARE v_used   INT;

  SELECT m.status, m.start_date, m.end_date, t.visit_limit
    INTO v_status, v_start, v_end, v_limit
  FROM membership m
    INNER JOIN membership_type t ON t.type_id = m.type_id
  WHERE m.membership_id = NEW.membership_id;

  IF v_status <> 'активен'
     OR DATE(NEW.entry_time) NOT BETWEEN v_start AND v_end THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Проход запрещён: абонемент не действует';
  END IF;

  IF v_limit IS NOT NULL THEN
    SELECT COUNT(*) INTO v_used
    FROM visit
    WHERE membership_id = NEW.membership_id;
    IF v_used >= v_limit THEN
      SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Проход запрещён: лимит посещений исчерпан';
    END IF;
  END IF;
END//

-- Триггер 3. Не даёт записать на занятие больше людей, чем есть мест,
-- и клиента без действующего абонемента
CREATE TRIGGER trg_booking_bi
BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
  DECLARE v_max   SMALLINT UNSIGNED;
  DECLARE v_date  DATE;
  DECLARE v_taken INT;
  DECLARE v_valid INT;

  SELECT max_places, DATE(start_time) INTO v_max, v_date
  FROM schedule
  WHERE schedule_id = NEW.schedule_id;

  SELECT COUNT(*) INTO v_taken
  FROM booking
  WHERE schedule_id = NEW.schedule_id AND status <> 'отменил';

  IF v_taken >= v_max THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Запись невозможна: свободных мест нет';
  END IF;

  SELECT COUNT(*) INTO v_valid
  FROM membership
  WHERE client_id = NEW.client_id
    AND status = 'активен'
    AND v_date BETWEEN start_date AND end_date;

  IF v_valid = 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Запись невозможна: нет действующего абонемента';
  END IF;
END//

-- Процедура 1. Заморозка абонемента на заданное число дней
CREATE PROCEDURE sp_freeze_membership (
  IN p_membership_id INT UNSIGNED,
  IN p_days          INT
)
BEGIN
  DECLARE v_status  VARCHAR(20);
  DECLARE v_allowed INT;

  SELECT m.status, t.freeze_days INTO v_status, v_allowed
  FROM membership m
    INNER JOIN membership_type t ON t.type_id = m.type_id
  WHERE m.membership_id = p_membership_id;

  IF v_status IS NULL THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Абонемент не найден';
  ELSEIF v_status <> 'активен' THEN
    SIGNAL SQLSTATE '45000'
```

```
      SET MESSAGE_TEXT = 'Заморозить можно только активный абонемент';
  ELSEIF p_days <= 0 OR p_days > v_allowed THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Срок заморозки превышает допустимый по тарифу';
  END IF;

  UPDATE membership
  SET status   = 'заморожен',
      end_date = DATE_ADD(end_date, INTERVAL p_days DAY)
  WHERE membership_id = p_membership_id;
END//

-- Процедура 2. Перевод просроченных абонементов в статус "истёк"
CREATE PROCEDURE sp_close_expired (
  IN  p_date  DATE,
  OUT p_count INT
)
BEGIN
  UPDATE membership
  SET status = 'истёк'
  WHERE end_date < p_date
    AND status = 'активен';

  SET p_count = ROW_COUNT();
END//

DELIMITER ;
```